CS795/895: Generative AI — Project Milestone 2 Report

Title: From Vertex AI to TinyLLM: Building Edge-Ready AI for Personalized Healthcare

Student: Anton Rasmussen

Course: CS795/895: Generative AI

1. Summary of Progress Since Milestone 1

In Milestone 1, I built and deployed a full Vertex AI workflow, established the motivation for combining cloud inference with local TinyLLM execution, and outlined a plan to experiment with model compression and federated adaptation.

For Milestone 2, I produced practical, measurable progress in two key areas:

1. LLM Compression Experiment (Dynamic Quantization)

- Implemented PyTorch dynamic INT8 quantization on `distilgpt2`
- Measured latency and storage footprint before and after quantization

2. Federated Learning Prototype (FedAvg)

- Implemented a minimal FedAvg simulation over synthetic client updates
- Validated aggregation logic for later integration with quantized models

Attempts to use AWQ and bitsandbytes encountered deprecation and compatibility issues, leading to a strategic pivot to stable PyTorch quantization.

2. Materials and Methods

Environment & Tools:

- Google Colab (Python 3.12)
- Libraries: transformers, torch, accelerate
- Model: distilgpt2

Baseline Inference Benchmark:

A helper function measured generation latency for 32 new tokens using a consistent prompt.

Dynamic INT8 Quantization:

PyTorch quantize_dynamic was applied to nn.Linear layers.

Model Size Measurement:

Serialized state_dicts were compared for compression evaluation.

Federated Learning Prototype:

A minimal FedAvg simulation averaged four synthetic client update vectors.

3. Results

Quantization results for distilgpt2:

Model | Latency (CPU, 32 tokens) | Size (MB) | Speedup | Compression

--------|-------------------------|-----------|---------|-------------

BL FP32 | 1.001 s | 312.50 MB | 1.00× | 1.00×

INT8 DQ | 1.188 s | 349.31 MB | 0.84× | 0.89×

Interpretation:

Dynamic quantization did not improve latency for a small model and slightly increased file size. This establishes a baseline and reinforces the **need for more advanced quantization methods for larger models**.

Federated Learning Prototype:

FedAvg aggregated the following synthetic client updates:

Client updates:

[[1.0, 2.0, 3.0],

[1.2, 1.9, 3.1],

[0.9, 2.1, 2.9],

[1.1, 2.2, 3.2]]

FedAvg global update:

[1.05, 2.05, 3.05]

4. Risks and Mitigations

Risk: AWQ incompatibility

Mitigation: Pivoted to stable PyTorch dynamic quantization.

Risk: bitsandbytes 4-bit unavailable in Colab

Mitigation: Deferred until a controlled environment is available.

Risk: Dynamic quantization underperforms for small models

Mitigation: Use this as a baseline; plan 4-bit or mixed-precision quantization for larger models.

5. Code & Reproducibility

See: Colab Project

All code for this milestone includes:

- Baseline model loading
- CPU latency benchmarking
- Dynamic INT8 quantization
- State-dict size comparison
- FedAvg prototype

Execution steps:

1. Install dependencies
2. Load model and benchmark
3. Apply quantization
4. Compare latency and size
5. Run FedAvg simulation

6. Roles and Contributions

This is an individual project. All design, implementation, experimentation, and documentation were completed by Anton Rasmussen.

7. Next Steps

- Explore modern quantization frameworks (torchao, vLLM llm-compressor)
- Scale compression experiments to larger models (Phi-3 Mini)
- Integrate quantized models into federated training loops
- Tie updated models back into a Vertex AI orchestration workflow